Solution: Find the Highest Altitude (easy)

Problem Statement

Given an array of integers representing a journey on a bike, wherein each number indicates a climb or descent in altitude, measured as a gain or loss of height.

The rider starts at altitude 0 and determines the highest altitude the biker reaches at any point during the journey.

Examples

Example 1

- **Input**: `[-5, 1, 5, 0, -7]`
- **Expected Output**: `1`
- **Justification**: The altitude changes are `[-5, -4, 1, 1, -6]`, where `1` is the highest altitude reached.

Example 2

- **Input**: `[4, -3, 2, -1, -2]`
- **Expected Output**: `4`
- **Justification**: The altitude changes are `[4, 1, 3, 2, 0]`, where `4` is the highest altitude reached.

Example 3

- **Input**: `[2, 2, -3, -1, 2, 1, -5]`
- **Expected Output**: `4`
- **Justification**: The altitude changes are `[2, 4, 1, 0, 2, 3, -2]`, where `4` is the highest altitude reached.

Solution

To solve this problem, we'll track the cumulative altitude as we traverse the array of altitude changes. We start with an initial altitude of zero. As we iterate through the array, we continually update the current altitude by adding the current change (which could be positive, negative, or zero) to it.

Simultaneously, we keep track of the highest altitude reached so far. This can be done by comparing the current altitude with the maximum altitude recorded after each update. Once we have gone through all the changes, the recorded maximum altitude is the answer. This approach efficiently calculates the highest altitude with a single pass through the array, ensuring optimal time complexity.

Here is the solution with detailed steps.

1. **Initialize Variables**:
   - Start by initializing two variables: `currentAltitude` and `maxAltitude`.
   - Set both `currentAltitude` and `maxAltitude` to 0, as the starting point is considered at sea level (zero altitude).
2. **Iterate Through the Array**:
   - Loop through each element of the given array of altitude changes.
   - For each element in the array, consider it as a change in altitude (gain or loss).
3. **Update Current Altitude**:
   - During each iteration, add the current element (altitude change) to `currentAltitude`.
4. **Check and Update Maximum Altitude**:
   - After updating `currentAltitude`, compare it with `maxAltitude`.
   - If `currentAltitude` is greater than `maxAltitude`, update `maxAltitude` with the value of `currentAltitude`.
   - This ensures that `maxAltitude` always holds the highest altitude reached so far.
5. **Continue Until the End of the Array**:
   - Continue the process of updating `currentAltitude` and checking/updating `maxAltitude` until you reach the end of the array.
6. **Return the Highest Altitude**:
   - After completing the iteration through the entire array, `maxAltitude` will hold the highest altitude reached during the journey.
   - Return `maxAltitude` as the final result.

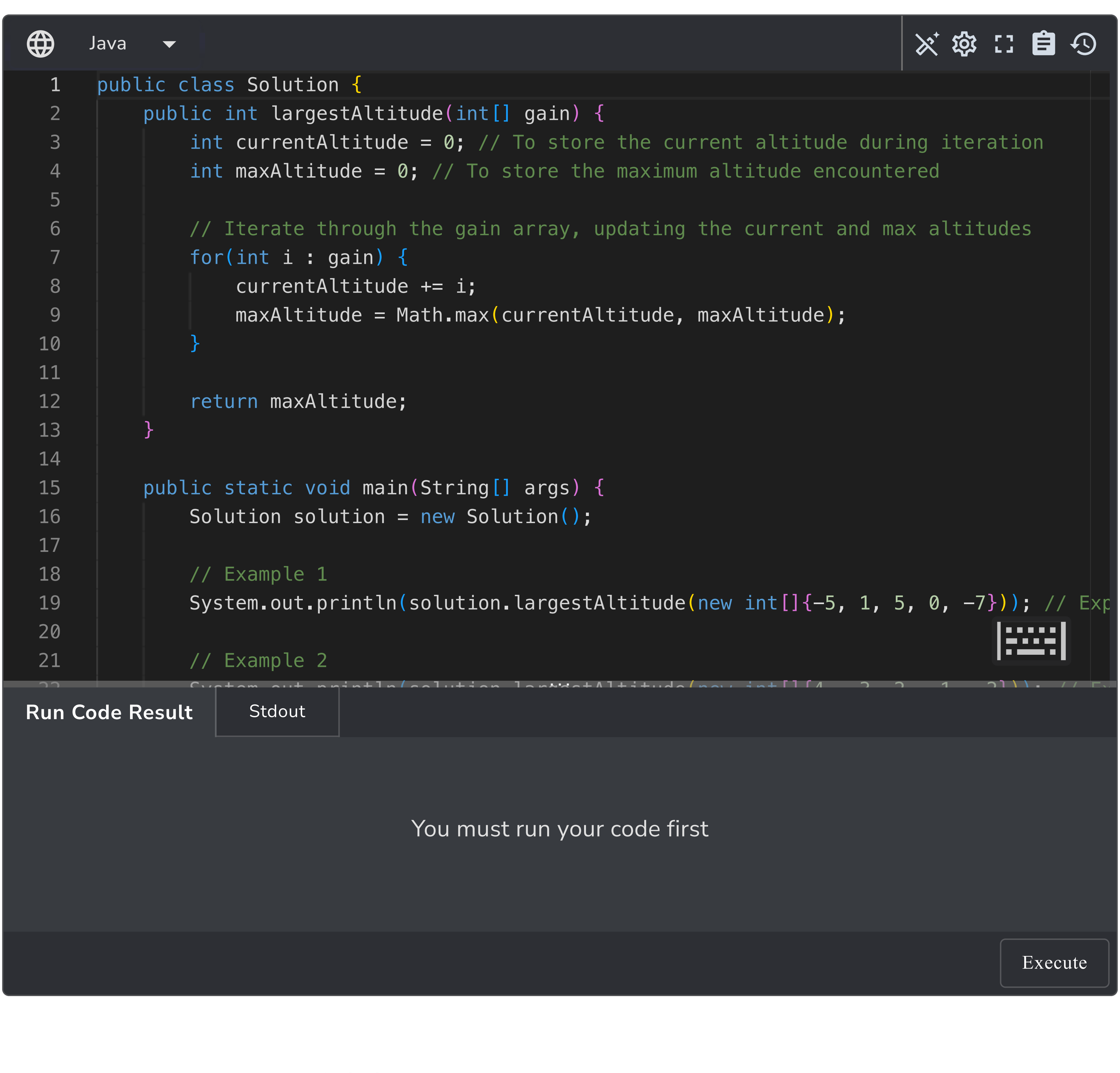
Algorithm Walkthrough

Given the input `[2, 2, -3, -1, 2, 1, -5]`, let's walk through the algorithm:

- Initialize `current_altitude = 0` and `max_altitude = 0`.
- Loop begins:
  - `i = 0`: Add `2` to `current_altitude` => `current_altitude = 2`. `max_altitude` is updated to `2`.
  - `i = 1`: Add `2` to `current_altitude` => `current_altitude = 4`. `max_altitude` is updated to `4`.
  - `i = 2`: Add `-3` to `current_altitude` => `current_altitude = 1`. `max_altitude` remains `4`.
  - `i = 3`: Add `-1` to `current_altitude` => `current_altitude = 0`. `max_altitude` remains `4`.
  - `i = 4`: Add `2` to `current_altitude` => `current_altitude = 2`. `max_altitude` remains `4`.
  - `i = 5`: Add `1` to `current_altitude` => `current_altitude = 3`. `max_altitude` remains `4`.
  - `i = 6`: Add `-5` to `current_altitude` => `current_altitude = -2`. `max_altitude` remains `4`.
- Loop ends.
- `max_altitude` which is `4` is returned as the output.

Code

Here is the code for this algorithm:



```java
public class Solution {
    public int largestAltitude(int[] gain) {
        int currentAltitude = 0; // To store the current altitude during iteration
        int maxAltitude = 0; // To store the maximum altitude encountered

        // Iterate through the gain array, updating the current and max altitudes
        for(int i : gain) {
            currentAltitude += i;
            maxAltitude = Math.max(currentAltitude, maxAltitude);
        }

        return maxAltitude;
    }

    public static void main(String[] args) {
        Solution solution = new Solution();

        // Example 1
        System.out.println(solution.largestAltitude(new int[]{-5, 1, 5, 0, -7})); // Exp

        // Example 2
```

You must run your code first

Time Complexity: O(n)

The algorithm's time complexity is linear, O(n), where "n" represents the length of the input list. This is because each element of the input list `gain` is visited once in a single loop to calculate the cumulative sum and find the maximum altitude. There are no nested loops or recursive function calls, keeping the time complexity linear.

Space Complexity: O(1)

The space complexity of the algorithm is constant, O(1). Regardless of the size of the input list, the algorithm only uses a few extra variables (`max_altitude` and `current_altitude`) to keep track of the current altitude and the maximum altitude achieved so far. These variables consume constant space, and no additional data structures, like arrays or matrices, that grow with the input are used, thus ensuring that the space usage is not dependent on the input size. This means the memory used by the algorithm does not increase with the input size, making it space-efficient for large inputs.

← Previous
Problem 4: Find The Highest Altitude(Easy)

Next →
Introduction To Matrix

☐ Mark as Completed